Session 06 — Loops

Python for Everyone · Lecture notes

> **How to read the code boxes:** grey boxes with a yellow bar contain Python code. The box labelled **Output** next to it (or below it) shows what Python prints when you run that code. In a Jupyter notebook, if the last line of a cell is just a value (like `titles` below), the notebook displays it; this is also shown as output.

Why loops?

- Repeating a command by copy-pasting it is cumbersome, hard to change and not very readable. And what if you need to repeat the code 30 or 3 million times?
- A **loop** runs a piece of code again and again. Python has two kinds: `while` and `for`. Loops are an essential part of every programming language, so practise them!

The `while` loop

- `while` repeats a block of code **as long as a condition is `True`**.

```
num = 1
while num <= 5:
    print(num, "sheep")
    num += 1
```

Output:

```
1 sheep
2 sheep
3 sheep
4 sheep
5 sheep
```

Anatomy of a `while` loop:

- `num <= 5` is the **condition**, checked before every round: `True` → run again, `False` → stop. The header line ends with a colon `:`.
- The **indented** block is the **body**: the code that is repeated.
- `num` is a **helper variable** that keeps track of where we are. Updating it (`num += 1`) is what eventually makes the condition `False`.

Infinite loop: if the condition never becomes `False`, the loop never stops. Here `num` goes down instead of up, so `num <= 10` stays `True`:

```
num = 1
while num <= 10:
    print(num)
    num -= 1
```

Output:

```
1
0
-1
-2
... (never stops)
```

- If this happens, interrupt Python: in Jupyter press the ■ (stop) button.

`break`

- `while True:` would run forever, unless the body contains `break`.
- `break` **exits the loop immediately**; the program continues after the loop.

```
correct_password = "cat"
while True:
    password = input("Enter password: ")
    if password == correct_password:
        print("Access granted!")
        break    # leave the loop
    print("Wrong password, try again.")
```

Output:

```
Enter password: dog
Wrong password, try again.
Enter password: cat
Access granted!
```

- Compared with `while password != correct_password:`, this version needs only one `input()`, and the code asking for the password sits right next to the check.

The `for` loop

- `for` goes through the elements of a container (e.g. a list) **one by one**.

```
books = [
    'Moby Dick (1851)',
    'The world according to Garp (1978)',
    'Networks: an Introduction (2018)',
    'Portraits of Empires (2023)'
]
for b in books:
    print(b[:-7])    # title without the year
```

Output:

```
Moby Dick
The world according to Garp
Networks: an Introduction
Portraits of Empires
```

Anatomy of a `for` loop:

- `for b in books:` is the **header**. `b` holds the **current element** (you choose its name); the **indented** body is repeated for every element.
- `books` can be any **iterable**: a container that can serve up its elements one by one (list, string, `range`, …).
- The same could be done with `while` and an index helper variable (`books[i]`, `i += 1`, see class notebook for example), but `for` is simpler.

A common pattern: start with an empty list and `append` to it in the loop:

```
titles = []
for b in books:
    titles.append(b[:-7])
titles
```

Output:

```
['Moby Dick', 'The world according to Garp', 'Networks: an Introduction', 'Portraits of Empires']
```

Iterating over other container objects

- A **string** gives its characters one-by-one:

```
for char in "Boo!":
    print(char)
```

Output:

```
B
o
o
!
```

- `range(n)` gives the whole numbers `0, 1, ..., n-1`:

```
for i in range(5):
    print(i, "sheep")
```

Output:

```
0 sheep
1 sheep
2 sheep
3 sheep
4 sheep
```

- `range(start, stop)`: like slicing, `start` is **included** and `stop` is **excluded**:

```
for i in range(3, 6):
    print(i)
```

Output:

```
3
4
5
```

- `range(len(L))` gives all valid indices of a list `L`: `for i in range(len(books)):` then use `books[i]` in the body.
- Later you will also iterate over dictionaries and over the rows of a file.

Example: finding the maximum

- Python has `max(nums)`, but let's write it ourselves.
- **Make a plan first:** how would you do it by hand? Remember the first number; look at each next number; if it is larger than the one you remember, remember it instead. At the end of the list you remember the largest number.

```
nums = [34, 42, 5, 8]
m = 0               # helper: largest value so far
for num in nums:
    if m < num:     # larger than the maximum so far?
        m = num     # update the maximum
m
```

Output:

```
42
```

Nested loops

- A loop can contain another loop. The inner loop runs completely for **every** round of the outer loop.

```
for i in range(2):
    for j in range(3):
        print(i, j)
```

Output:

```
0 0
0 1
0 2
1 0
1 1
1 2
```

Debugging

- Mistakes in loops are normal. To find them, **print out variables inside the loop** to see what happens in each round.
- In class we fixed a broken version of the sorting code: it removed the number from `sorted_nums` instead of `nums`, so `nums` never got shorter and the `while` loop never stopped.

Common mistakes

- **Missing colon:** `while num <= 5` without `:` at the end gives `SyntaxError: expected ':'`. The same goes for `for`.
- **Missing indentation:** the body must be indented, otherwise you get `IndentationError: expected an indented block after 'while' statement ...`.
- **Indenting too much:** code meant to run *after* the loop (e.g. `print("Blast off!")`) must not be indented, otherwise it runs in every round.
- **Forgetting to update a helper variable** (or changing it the wrong way). If you get an infinite loop, press ■ to stop it.
- **Off by one with `range`:** `range(5)` is `0, 1, 2, 3, 4`, not `1, ..., 5`. For `1, ..., 5` use `range(1, 6)`.

Summary

Term / code	Meaning
`while condition:`	repeat the body as long as the condition is `True`
`for x in container:`	repeat the body for every element; `x` is the current one
`IndentationError`	the body of a loop is not indented
helper variable	keeps track of the state (counter, maximum so far, …)
infinite loop	the condition never becomes `False`; stop it with ■
`while True:` + `break`	loop until `break` exits it
iterable	anything a `for` loop can go through: list, string, `range`
`range(n)`	`0, 1, ..., n-1`
`range(a, b)`	`a, a+1, ..., b-1`
`range(len(L))`	all indices of the list `L`